ISSN 0361-7688, Programming and Computer Software, 2026, Vol. 52, No. 1, pp. 39–49. © Pleiades Publishing, Ltd., 2026.
Russian Text © The Author(s), 2026, published in Programmnaya Ingeneria, 2025, Vol. 16, No. 11.

Software Verification Witnesses Thoroughness

V. O. Mordan[a,*] and V. S. Mutilin[a,b,]**

[a] *Ivannikov Institute for System Programming, Russian Academy of Sciences, Moscow, 109004 Russia*

[b] *Moscow Institute of Physics and Technology (National Research University), Dolgoprudny, Moscow oblast, 141700 Russia*

**e-mail: mordan@ispras.ru*

***e-mail: mutilin@ispras.ru*

Received May 29, 2025; revised May 29, 2025; accepted July 8, 2025

Abstract—Software verification is a resource-intensive process that combines automated program analysis with manual evaluation of results. While modern methodologies and cloud technologies have expedited the automated phase, manual analysis remains a critical bottleneck, particularly for large-scale software systems, due to the need for specialized developer expertise. Verification results often include warnings about potential bugs, which must be either fixed or classified as false alarms, and correctness proofs intended to demonstrate the absence of defects. This paper addresses these challenges by formally introducing the concept of witness thoroughness in software verification. We evaluate this concept using results from tools that participated in the Competition on Software Verification, demonstrating its practical applicability to real-world verification benchmarks. Furthermore, we show how witness thoroughness can serve as a meaningful metric for comparing verification tools and provide recommendations to improve validation rates. Finally, we identify which error trace elements are essential for effective visualization of witnesses.

Keywords: software verification, thoroughness, verification results, validation, error trace

DOI: 10.1134/S0361768826700052

1. INTRODUCTION

Software verification serves as a formal method for automatic examining a program's source code without executing it, traversing all possible program paths. The objective is to prove program correctness against specified properties under corresponding assumptions, in contrast to testing techniques and bug finding methods like static analysis. Software verification accommodates various property types, such as unreachability of error functions or absence of data races. While its primary goal is proving correctness, these methods excel at detecting all property violations [1] including those, which are hard to reproduce with tests, making them particularly suitable for critical software systems like operating systems, online banking, and avionics.

The ongoing advancements in software verification are evident in the continuous improvement of both efficiency and effectiveness, as demonstrated by the annual Competition on Software Verification (SV-COMP) [2–4]. This progress has enabled the successful application of verification tools to an ever-expanding range of software systems. For example, in SV-COMP'23, 52 state-of-the-art verification tools were evaluated on benchmarks comprising 23 805 C programs and 586 Java programs, each with known results. The benchmarks were categorized based on the property being verified, and each participating tool addressed its tasks under specified resource constraints (15 GB of RAM, 8 CPU cores, and 15 minutes of CPU time).

The tools generate results in the form of witnesses in GraphML format (version 1.0) [5], which represent either correctness proofs or error traces. These witnesses serve a dual purpose: they enable software developers to manually analyze and fix bugs, and they enable automatic validation by specialized tools known as witness validators, thereby increasing confidence in verification results.

However, manual analysis becomes necessary when no validator is able to confirm the validity of a witness or when inaccuracies arise. For instance, false alarms may result from incorrectly specified requirements, such as misdefined properties, or misapplied tool assumptions, requiring users to distinguish these from actual bugs.

A notable challenge in software verification is validating results for complex systems like Linux device drivers, where the process is both resource-intensive and difficult. Many witnesses generated by tools such as ESBMC-kind [6] and Crux [7] for Linux device drivers have never been validated. Without manual analysis, it is impossible to determine whether these

witnesses are incorrect or simply missing crucial elements.

Manual analysis played a crucial role in the Google Summer of Code Linux driver verification efforts, where 62 potential bugs were initially reported. However, after an extensive three-month manual review, only two of these issues were confirmed[1]. Similar outcomes have been observed in other projects[2]. This highlights a critical issue: users require validated results that contain all the necessary elements to facilitate effective bug identification. As a result, the practical success of software verification relies heavily on the completeness and quality of generated witnesses.

For both validation and manual analysis, it is essential that a witness includes key elements such as function calls and assumptions. However, some verification tools produce witnesses that lack these crucial details, making validation unreliable and manual analysis impractical.

In this paper, we address the following research question: which witness elements are most critical for the validation process and manual analysis? We focus primarily on error trace validation, leaving the analysis of correctness proofs and manual error trace investigation for future work.

Our central hypothesis is that including key elements in a witness significantly enhances validation. Our objective is to identify these critical components, which will form the foundation of our concept of witness thoroughness. This will allow us to assess the thoroughness of witnesses generated by different verification tools and evaluate how easily they can be validated.

By defining and measuring witness thoroughness, we aim to develop an automated method for determining whether a witness provides a complete error trace – one that enables developers to either fix the corresponding bug or dismiss it as a false alarm – or if it is incomplete and unsuitable for manual analysis.

This paper makes the following contributions:

• We formally define the concept of witness thoroughness in software verification.

• We evaluate witness thoroughness for tools participating in the Competition on Software Verification by analyzing their produced results (error traces only).

• We demonstrate how witness thoroughness can serve as a metric for comparing verification tools and offer practical recommendations for improving validation rates.

[1] Development of environment model specifications for static verification of Linux Kernel: http://linuxtesting.org/31-08-2020.

[2] For example, http://linuxtesting.org/16-08-2016, http://linuxtesting.org/27-08-2017, http://linuxtesting.org/28-08-2017, http://linuxtesting.org/13-08-2018, http://linuxtesting.org/28-08-2020, http://linuxtesting.org/20-08-2021, and http://linuxtesting.org/23-08-2021.

The following section provides essential definitions and an overview of the SV-COMP benchmarks. In section 3, we formally define witness thoroughness and evaluate it on the SV-COMP benchmarks. Section 4 presents a comparative analysis of SV-COMP tools, highlighting their performance based on witness thoroughness.

2. BACKGROUND

We examine a software system written in the C language that requires verification against certain properties. The checked **properties** are either generic for the C language (such as memory safety, absence of overflows, absence of data races, and program termination) or unreachability of the specific function (like `abort()` or error()).

The **verification task** comprises the program source code, a component of the software system being analyzed, an entry point (e.g., the main() function), and specified properties for examination. Additionally, a verification task may contain resource constraints and various configuration options. A **software verifier** is a tool that automatically solves a given verification task and offers a response, indicating whether a particular property is violated in a program or not.

The **verification result** serves as a formal response to a given verification task, typically falling into one of three possible outcomes:

• **TRUE**. The software verifier has demonstrated that a specified property cannot be violated in the given program and has provided a correctness proof. If the proof is incorrect, this situation is labeled as a **missed bug**.

• **FALSE**. The software verifier has identified property violations in the given program and has supplied error traces. If a particular error trace does not correspond to a real bug in the program, it is considered a **false alarm**.

• **UNKNOWN**. The software verifier was unable to resolve the given verification task due to resource exhaustion or errors in the tool itself. The log generated can be valuable for verifier developers as a bug report.

An **error trace** is a one or more paths of operations in a program's source code from an entry point to a property violation. On the other hand, a **correctness proof** is a graph depicting all potential program executions from an entry point, devoid of any property violation [8]. The terms "error trace" and "correctness proof" collectively refer to **witnesses**. Going forward, we exclusively examine witnesses adhering to the SV-COMP common format [5], aligning with the standards for software verifiers engaged in competitions.

A **validator** is a tool [9] designed to take a verification task and its corresponding witness as input, determining whether the provided result witness is accurate or not. Combinations of validators can be employed to

enhance the confidence in verification outcomes. In terms of manual result analysis, validation automatically reduces the number of inadequate witnesses.

The verification process is illustrated in Fig. 1. Each verification task is solved by a software verifier, and the obtained results can subsequently be validated. This stage is fully automated, enabling efficient use of computational resources through fast validation tools and parallel execution of diverse verification tasks in cloud environments.

Following this automated phase, manual analysis becomes essential. At first, all identified error traces must be classified as either false alarms or real bugs, with the latter requiring attention from software developers. This highlights the importance of presenting witnesses in a form that is easily understandable to developers. When the verification goal is to prove correctness, the analysis shifts to the inspection of proofs to ensure that no bugs were missed. Such examination is inherently more complex due to the intricate structure of proofs and requires significant user expertise. Consequently, reducing the effort needed for results analysis in this context is considerably more challenging.

This paper addresses the following research question: which elements of witnesses are essential for successful validation and may potentially enhance human comprehension during manual analysis?

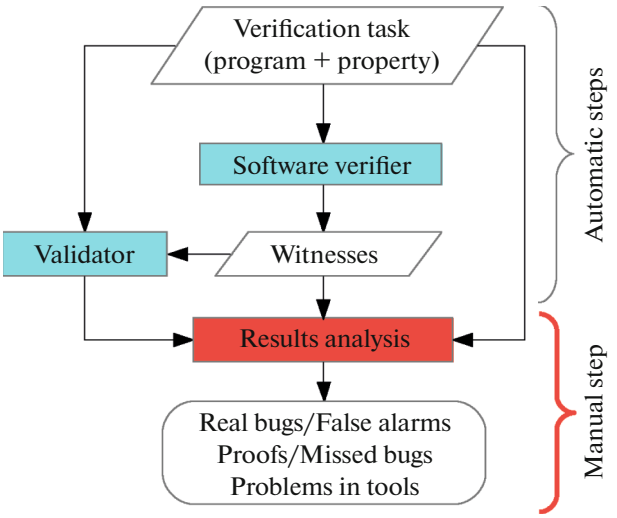

Fig. 1. The verification process.

2.1. SV-COMP Verification Tasks

The competition benchmarks in SV-COMP are grouped into distinct verification categories, each targeting a specific software property to be checked. The properties included in SV-COMP'23 are as follows:

- *ReachSafety*: Verifies whether certain function calls are unreachable within the program, ensuring that specific undesirable or erroneous states cannot be reached.
- *MemSafety*: Ensures the absence of memory-related errors, such as invalid pointer dereferences, and use-after-free violations.
- *ConcurrencySafety*: Addresses challenges in concurrent programs, such as data races, deadlocks, and other thread-related issues, ensuring proper synchronization and thread-safe behavior.
- *NoTermination*: Determines whether a program terminates for all possible inputs, thereby avoiding infinite loops or non-terminating behaviors.
- *NoOverflow*: Validates that no arithmetic operations exceed the allowable numerical range, preventing unintended behavior caused by overflow or underflow.

Each category consists of a set of verification tasks, where each task represents a program with a specific property to be verified. The verification tasks are provided with known expected results, enabling precise evaluation of the tools. These tasks range in complexity and domain, encompassing general-purpose applications as well as domain-specific software like Linux device drivers.

Each participating tool attempts to solve the given tasks within predefined resource constraints, producing a verification result for each task. The verification result must match the known expected outcome and witness provided by the tool must be successfully validated by an independent validation process. The results of the competition provide insights into the strengths and limitations of each tool, driving advancements in the field of software verification.

The validation process in SV-COMP is well-established – certain verifiers (such as CPAchecker [10] or UAutomizer [11]) with specific configurations take a source code and a witness, and either confirm it or not [9]. Based on SV-COMP statistics, we can calculate the validation rate for each tool and property, which can be considered a formal measure of the quality of the produced witnesses. However, this information is only accessible through the validation process, which also requires significant resources. Additionally, the current validation process does not provide insights into why a witness was rejected or which elements it might have been missing.

Another, more intuitive approach to evaluating witnesses is through visualization. Currently, SV-COMP does not offer any standardized guidance on witness visualization, leaving this task to the discretion of individual verifiers or external tools. Projects like LDV Tools [12] and Klever [13] facilitate the visualization of verification results, enabling users to manually analyze and compare witnesses. This approach helps

users better understand the root causes of bugs, although it relies heavily on manual effort.

3. WITNESSES THOROUGHNESS

3.1. Witness Structure

Initially, let's examine the structure of a witness in the GraphML common format (version 1.0) [5]. It comprises a header containing auxiliary information such as the path to source files, property details, witness type, source code language, tool used, timestamp, and more. The witness also includes edges and nodes forming a graph, which represents the internal structure of an error trace or a correctness proof. Each node may have additional tags:

• **Entry**. Represents the initial node of the witness graph.

• **Violation**. Indicates a property violation in an error trace.

• **Sink**. Marks unexplored paths in conditions within error traces.

• **Invariant**. Specifies an invariant as a C expression, along with its scope, for correctness proofs.

Edges represent operations linked to their respective source lines. We categorize all types of elements into the following groups:

• **Function Calls** or **FC** (tags: *enterFunction* and *returnFromFunction*) serve as a foundational structural element for error traces. It proves particularly valuable as a comprehensive sequence from an entry point to a property violation is necessary for analysis.

• **Conditions** or **C** (tag: *control*) indicate which branch was utilized, with values of either *condition-true* (true evaluation) or *condition-false* (false evaluation). Unexplored conditions may lead to sink nodes in error traces.

• **Assumptions** or **A** (tag: *assumption*) specify a C expression considered true within a given scope. It is used to evaluate variable values, function arguments, and function return values.

• **Thread specifics** or **TS**. The tag *threadId* provides information about the current thread identifier for each edge, while *createThread* marks the creation of a new thread. These tags are mandatory for analyzing witnesses in parallel programs.

• **Loop heads** or **LH** (tag: *enterLoopHead*). Mark the entry points of loops, constraining possible transitions in the program's control flow.

An edge may contain the tag *sourcecode* with corresponding source code or a link to source code (tags: *startline*, *endline*, *startoffset*, *endoffset*), facilitating the restoration of operations. Such tags are useful for restoring each step during bug reproduction and thus are crucial for visualization purposes.

It is important to emphasize that the elements described are not universally essential for validating a bug across all properties. For example, the *MemSafety* property typically requires operations related to the given memory, such as allocation, assignments, and freeing. In such cases, only specific types of operations, like assumptions and, possibly, function calls, are essential for manual analysis. Additionally, including all operations in an error trace may complicate it, as the user only needs to see the relevant operation for the identified bug. However, in general cases (especially for complex tasks, as illustrated in the SV-COMP software systems category), including all elements may offer better opportunities for validation.

3.2. Formal Definition of Witness Thoroughness

Consider a set of verification tasks $\mathcal{T}$, along with their corresponding verification results produced by a given verifier for a property $\mathcal{P}$. In this context, we focus exclusively on error traces within the verification results. Let $\mathcal{W}$ denote the set of error traces corresponding to these verification results. The **validation rate**, denoted as $\mathcal{R}$, is defined as the percentage of successfully validated error trace witnesses out of the total number of error trace witnesses:

$$\mathcal{R} = \frac{|\mathcal{W}_{\text{validated}}|}{|\mathcal{W}|} \times 100,$$

where $\mathcal{W}_{\text{validated}} \subseteq \mathcal{W}$ represents the subset of validated witnesses.

As discussed in the previous section, each witness $\mathcal{W}$ contains elements categorized into five distinct types: FC, C, A, TS, and LH. A witness $w \in \mathcal{W}$ is considered to contain a specific type of element if it includes more than one instance of that type. If a witness contains only a single, trivial element (e.g., a single FC for an entry point that is identical across all tools), it is not considered as genuinely containing that type.

A *relevant type of element* is defined as an element type (one of FC, C, A, TS, LH) that contributes to an increase in the validation rate $\mathcal{R}$ when present in the witness. Let $\mathcal{E}_w \subseteq \{\text{FC}, \text{C}, \text{A}, \text{TS}, \text{LH}\}$ denote the set of element types present in witness w, and let $\mathcal{E}_{\text{relevant}} \subseteq \{\text{FC}, \text{C}, \text{A}, \text{TS}, \text{LH}\}$ represent the set of relevant element types. The specific criteria for determining relevant elements will be discussed in detail in the next section.

The **witness thoroughness** for a given set of witnesses $\mathcal{W}$, denoted as $WT(\mathcal{W})$, is defined as the maximum percentage of relevant element types present in any witness within $\mathcal{W}$:

$$WT(\mathcal{W}) = \max_{w \in \mathcal{W}} \frac{|\mathcal{E}_w \cap \mathcal{E}_{\text{relevant}}|}{|\mathcal{E}_{\text{relevant}}|} \times 100.$$

Informally, witness thoroughness measures how well a tool includes the key elements necessary for validation. Since not all verification tasks are equally complex, some may lack conditions or other important elements entirely. To account for this, we focus on the witness with the highest thoroughness score, ensuring that overly simplistic tasks do not underestimate the tool's ability to produce informative witnesses. The normalization by relevant elements ensures that the measure remains meaningful across different properties.

Witness thoroughness serves as a useful metric for comparing different tools with respect to a specified property P and a set of verification tasks. In other words, witness thoroughness provides an indication of how well a witness facilitates validation without the need to execute a resource-intensive validation process. This makes it a valuable measure for evaluating tool effectiveness while minimizing computational overhead.

3.3. Evaluation of Relevant Elements for SV-COMP Properties

Next, we evaluated the presence of relevant witness elements for SV-COMP properties using the SV-COMP results as a training dataset [14]. This dataset included witnesses produced by 43 verifiers across five properties, along with their corresponding validation outcomes. For this evaluation, we considered all verifiers that participated in the respective SV-COMP'23 category and produced at least ten error traces. A complete description of the evaluation procedure and instructions for reproducing the results are provided in the artifact [15].

Our primary research question was: which types of elements FC, C, A, TS, and LH are the most important for validation across each property? To address this, we developed a linear regression model to analyze the relationship between these five witness elements and the validation rate.

To conduct this analysis, we employed automatic relevance determination (ARD) regression [16]. ARD regression is a type of Bayesian regression technique that is particularly useful in high-dimensional feature spaces. By incorporating sparsity-promoting priors, ARD automatically determines the relevance of each feature, allowing the model to weight the contribution of each witness element (FC, C, A, TS, and LH) based on their impact on the validation rate. This approach is advantageous because it mitigates the risk of overfitting while ensuring that only the most informative features are retained in the model, thus enhancing the interpretability and reliability of the results. Also it favors simplicity, meaning the model inherently prefers solutions that use fewer features unless the data strongly suggests their relevance.

Table 1. Normalized ARD regression coefficients showing the relationship between each element and validation rate, with high-magnitude (>0.3) positive coefficients bolded

Property	FC	TS	A	C	LH
ReachSafety	**1**	0	**0.95**	**1**	**1**
MemSafety	0.112	0.266	**1**	0.112	0
ConcurrencySafety	0	**0.356**	**1**	0	0
NoTermination	–0.202	0	**0.629**	**0.798**	0
NoOverflow	0	–0.089	**0.911**	0	0

Using ARD regression, we examined the relationship between the witness elements and the validation rate across different SV-COMP properties. The results, presented in Table 1, offer insights into which elements are most critical for the successful validation of witnesses. For clarity, we normalized the coefficients and focused on positive coefficients with high magnitudes [16] (greater than 0.3), as these indicate a direct relationship with validation rate. Negative coefficients were not considered because they suggest an inverse relationship, implying that adding such elements would reduce the validation rate, which is not relevant to our analysis.

The conducted evaluation demonstrates how the thoroughness can be computed for any generic property, provided that a set of validated error traces is available. At the same time, verification tools with identical thoroughness values may produce different error traces (for example, containing different assumption elements), which results in varying validation rates – certain crucial elements may be missing, leading to unsuccessful validation. Thus, although thoroughness is an important prerequisite for achieving a high validation rate, it is not its sole determining factor.

Therefore, the relevant elements for SV-COMP properties can be identified as follows:

1. *ReachSafety*: function calls, assumptions, conditions, loop heads.
2. *MemSafety*: assumptions.
3. *ConcurrencySafety*: assumptions, thread specifics.
4. *NoTermination*: assumptions, conditions.
5. *NoOverflow*: assumptions.

4. ASSESSING WITNESS THOROUGHNESS FOR SV-COMP TOOLS

In this section we determined the witness thoroughness for all tools, that participated in SV-COMP'23 for the verification of C programs for each property separately. Two software verifiers (CoVeriTeam-Verifier-AlgoSelection and CoVeriTeam-Veri-

Table 2. Comparison of thoroughness with the validation rate for the *ReachSafety* property in SV-COMP'23 (3173 tasks). Relevant elements (FC, A, C, LH) are bolded for clarity

SV-COMP Tool	**FC**	TS	**A**	**C**	**LH**	Validation rate (%)	Thoroughness (%)
Bubaak	**0**	0	**1**	**0**	**0**	62.07	25
CBMC	**0**	1	**1**	**0**	**0**	5.61	25
CPA-BAM-BnB	**1**	0	**1**	**1**	**1**	83.10	100
CPA-BAM-SMG	**1**	0	**1**	**1**	**1**	85.07	100
CPAchecker	**1**	1	**1**	**1**	**1**	89.58	100
Crux	**0**	0	**1**	**0**	**0**	0.13	25
ESBMC-kind	**0**	1	**1**	**0**	**0**	21.74	25
Graves-CPA	**1**	1	**1**	**1**	**1**	84.51	100
Graves-Par	**1**	1	**1**	**1**	**1**	93.33	100
Infer	**0**	0	**0**	**0**	**0**	0.00	0
PeSCo-CPA	**1**	1	**1**	**1**	**1**	80.28	100

Table 3. Comparison of thoroughness with the validation rate for the *MemSafety* property in SV-COMP'23 (3202 tasks), with column for relevant elements bolded

SV-COMP Tool	FC	TS	A	C	LH	Validation rate (%)	Thoroughness (%)
2LS	0	0	**1**	0	0	61.29	100
Bubaak	0	0	**1**	0	0	97.94	100
CBMC	0	1	**1**	0	0	67.89	100
CPAchecker	1	1	**1**	1	1	96.26	100
DIVINE	0	0	**1**	0	0	0	100
ESBMC-kind	0	1	**1**	0	0	90.47	100
Graves-CPA	1	1	**1**	1	1	95.70	100
Graves-Par	1	1	**1**	1	1	95.89	100
PeSCo-CPA	1	1	**1**	1	1	99.32	100
Symbiotic	0	1	**1**	0	1	98.85	100
UAutomizer	1	1	**1**	1	1	94.58	100
UKojak	1	0	**1**	1	0	96.06	100
UTaipan	1	1	**1**	1	0	94.32	100

fier-ParallelPortfolio) were excluded from the comparison because they are composed of other tools that are already included in the comparison. In this evaluation, we exclusively considered error traces. The results with presented witness elements, determined thoroughness and validation rate can be found in tables from 2 to 6.

The evaluation demonstrates that witness thoroughness generally correlates with higher validation rates across most properties. However, there are notable exceptions, such as EBF [17] for the *ConcurrencySafety* property, where high validation rates are achieved despite low thoroughness.

Different properties exhibit varying levels of dependence on witness thoroughness. For instance, properties like *ReachSafety* and *NoTermination* display a strict dependency on thoroughness, while others, such as *MemSafety* and *ConcurrencySafety*, achieve moderate

Table 4. Comparison of thoroughness with the validation rate for the *ConcurrencySafety* property in SV-COMP'23 (2865 tasks), with columns for relevant elements bolded

SV-COMP Tool	FC	TS	A	C	LH	Validation rate (%)	Thoroughness (%)
CBMC	0	**1**	**1**	0	0	84.81	100
CPALockator	1	**1**	**1**	1	1	26.51	100
CPAchecker	1	**1**	**1**	1	1	100.00	100
Cseq	1	**1**	**1**	0	0	25.89	100
Dartagnan	0	**1**	**0**	0	0	90.24	50
Deagle	0	**1**	**1**	0	0	88.71	100
DIVINE	0	**0**	**1**	0	0	80.87	50
EBF	0	**0**	**0**	0	0	83.43	0
ESBMC-incr	0	**1**	**1**	0	0	79.41	100
ESBMC-kind	0	**1**	**1**	0	0	89.73	100
Graves-CPA	1	**1**	**1**	1	1	99.23	100
Graves-Par	1	**1**	**1**	1	1	100.00	100
Infer	0	**0**	**0**	0	0	0.00	0
Lazy-CSeq	1	**1**	**1**	1	0	94.89	100
LF-checker	0	**1**	**1**	0	0	85.31	100
PeSCo-CPA	1	**1**	**1**	1	1	100.00	100
PIChecker	1	**0**	**1**	1	1	98.14	50
Symbiotic	0	**1**	**1**	0	1	92.73	100
UAutomizer	1	**1**	**1**	1	1	94.68	100
UGemCutter	1	**1**	**1**	1	0	96.47	100
UTaipan	1	**1**	**1**	1	0	95.76	100

Table 5. Comparison of thoroughness with the validation rate for the *NoTermination* property in SV-COMP'23 (1809 tasks), with columns for relevant elements bolded

SV-COMP Tool	FC	TS	A	C	LH	Validation rate (%)	Thoroughness (%)
2LS	0	0	**1**	**0**	0	69.08	50
Bubaak	0	0	**1**	**0**	0	34.78	50
CPAchecker	1	1	**1**	**1**	1	97.01	100
Graves-CPA	1	1	**1**	**1**	1	81.15	100
PeSCo-CPA	1	1	**1**	**1**	1	98.26	100
Symbiotic	0	1	**1**	**0**	1	52.96	50
UAutomizer	1	1	**1**	**1**	1	98.24	100
VeriFuzz	0	0	**0**	**1**	1	71.34	50

validation rates even with reduced thoroughness. This indicates that these properties may rely more on specific witness elements rather than overall completeness.

Tools like CPAchecker, Graves-CPA [18], and PeSCo-CPA [19] consistently perform well across all properties, achieving both high thoroughness and validation rates. Their consistent performance underscores their suitability as reliable options for general-purpose verification tasks. Conversely, tools with low thoroughness, such as Crux [7] and Infer [20], exhibit notably poor validation rates across most properties due to their provision of empty witnesses.

The presented tables offer actionable insights into which witness elements could be added to improve validation rates for specific tools. For example, while

Table 6. Comparison of thoroughness with the validation rate for the *NoOverflow* property in SV-COMP'23 (6618 tasks), with the column for relevant elements bolded

SV-COMP Tool	FC	TS	A	C	LH	Validation rate (%)	Thoroughness (%)
2LS	0	0	**1**	0	0	95.70	100
Bubaak	0	0	**1**	0	0	94.67	100
CBMC	0	1	**1**	0	0	62.14	100
CPAchecker	1	1	**1**	1	1	100	100
Crux	0	0	**1**	0	0	95.05	100
ESBMC-kind	0	1	**1**	0	0	66.69	100
Frama-C-SV	0	0	**0**	0	0	0	0
Graves-Par	1	1	**1**	1	1	2	100
Infer	0	0	**0**	0	0	0	0
Pinaka	0	0	**1**	0	0	100	100
Symbiotic	0	1	**1**	0	1	100	100
UAutomizer	1	1	**1**	1	1	100	100
UKojak	1	0	**1**	1	0	100	100
UTaipan	1	1	**1**	1	0	100	100
VeriFuzz	0	0	**1**	0	1	90.81	100

the CBMC [21] tool performs exceptionally well for the *ConcurrencySafety* property (achieving an 84.81% validation rate), it struggles significantly with more complex *ReachSafety* tasks due to missing witness elements, such as function calls and conditions. Similarly, Symbiotic [22] excels in the *MemSafety*, *ConcurrencySafety*, and *NoOverflow* properties (achieving 100% thoroughness) but underperforms in *NoTermination* (50% thoroughness) due to the absence of required conditions.

Our evaluation demonstrates a reliable approximation of the validation rates. However, we observe that tools with identical levels of thoroughness can produce different traces, leading to variations in validation rates. This finding highlights that while thoroughness is a necessary condition for achieving high validation rates, it is not a sufficient condition on its own.

Table 7. Thoroughness comparison among the winners of SV-COMP'23 for the *ReachSafety* property (based on SoftwareSystems category)

SV-COMP Tool	FC	A	C	LH	Thoroughness
I. Symbiotic	0	1	0	1	50
II. Bubaak	0	1	0	0	25
III. Mopsa	no error traces				0

4.1. Witness Thoroughness Analysis for SV-COMP Winners

Another question that arises here is: do the winners always exhibit the highest thoroughness? Tables from 7 to 11 presents a comparison of thoroughness among the winners across all SV-COMP'23 properties.

For properties like *MemSafety*, *NoOverflow*, and *ConcurrencySafety* there is a clear and direct correlation between winning tools and 100% thoroughness – winning tools consistently achieve both high validation rates and maximum thoroughness. However, for properties such as *ReachSafety* and *NoTermination*, no such correlation is observed. Winners in these categories often achieve only moderate or minimal thoroughness (e.g., Mopsa [23] with 0% thoroughness for *ReachSafety*).

A closer analysis reveals that such tools gain higher scores by producing more validated correctness proofs. For instance, Mopsa did not produce a single error trace, while Symbiotic managed to provide only five error traces for *ReachSafety*. Since our evaluation focused exclusively on error traces, this scoring approach may have influenced the results.

Table 8. Thoroughness comparison among the winners of SV-COMP'23 for the *MemSafety* property

SV-COMP Tool	A	Thoroughness
I. Symbiotic	1	100
II. CPAchecker	1	100
III. UTaipan	1	100

Table 9. Thoroughness comparison among the winners of SV-COMP'23 for the *ConcurrencySafety* property

SV-COMP Tool	TS	A	Thoroughness
I. Deagle	1	1	100
II. UAutomizer	1	1	100
III. UGemCutter	1	1	100

Table 10. Thoroughness comparison among the winners of SV-COMP'23 for the *NoTermination* property

SV-COMP Tool	A	C	Thoroughness
I. VeriFuzz	1	0	50
II. UAutomizer	1	1	100
III. 2LS	1	0	50

Table 11. Thoroughness comparison among the winners of SV-COMP'23 for the *NoOverflow* property

SV-COMP Tool	A	Thoroughness
I. UAutomizer	1	100
II. UKojak	1	100
III. UTaipan	1	100

Another point is that the validation rate heavily depends on the given set of witnesses – if the set is too easy to validate, the observed validation rate may not accurately reflect the tool's overall effectiveness. This suggests that the validation rate alone may not be a universally reliable criterion for assessing performance across all tasks for a given property.

These findings underscore the value of the thoroughness metric, which formally evaluates witness quality, and incorporating such a metric into SV-COMP could be highly beneficial for SV-COMP.

5. CONCLUSIONS

In this paper, we formally defined the concept of witness thoroughness in software verification, which is based on identifying key witness elements relevant to the property being verified. Using benchmarks from the Competition on Software Verification, we conducted a detailed evaluation of these key elements and assessed the thoroughness of all tools participating in the competition. Our analysis revealed a clear dependency between witness thoroughness and the validation rate of verification results. This demonstrates how witness thoroughness can serve as a practical metric for comparing verification tools, introducing a new criterion for evaluating witness quality. Additionally, we provided specific recommendations for improving thoroughness and, consequently, validation rates for each tool and property.

In the future, our work will focus on witness visualization as a practical application of the thoroughness metric. Presenting key witness elements in a clear and concise manner is essential for effective manual analysis. The thoroughness metric can guide the balance – ensuring that critical elements are highlighted while avoiding an overload of irrelevant details that might overwhelm users. This approach will represent a significant step toward making witnesses more accessible and interpretable in a human-readable format.

Additionally, we plan to extend the concept of witness thoroughness to encompass correctness proofs. Unlike error traces, which follow a simple sequential structure, correctness proofs have a non-linear graph structure, making their analysis significantly more complex. This expansion will likely require the development of additional evaluation criteria to assess their quality and determine whether they can identify instances of missing bugs.

FUNDING

This work was supported by ongoing institutional funding. No additional grants to carry out or direct this particular research were obtained.

CONFLICT OF INTEREST

The authors of this work declare that they have no conflicts of interest.

REFERENCES

1. Mordan, V. and Novikov, E., Minimizing the number of static verifier traces to reduce time for finding bugs in Linux kernel modules, *Proceedings of the 8th Spring/Summer Young Researchers' Colloquium on Software Engineering* (*SYRCoSE 2014*), 2014. https://doi.org/10.15514/SYRCOSE-2014-8-5

2. Beyer, D., Competition on software verification and witness validation: SV-COMP 2023, *Tools and Algorithms for the Construction and Analysis of Systems*, Sankaranarayanan, S. and Sharygina, N., Eds., Lecture Notes in Computer Science, vol. 13994, Cham: Springer, 2023, pp. 495–522. https://doi.org/10.1007/978-3-031-30820-8_29
3. Beyer, D., Progress on software verification: SV-COMP 2022, *Tools and Algorithms for the Construction and Analysis of Systems*, Fisman, D. and Rosu, G., Eds., Lecture Notes in Computer Science, vol. 13244, Cham: Springer, 2022, pp. 375–402. https://doi.org/10.1007/978-3-030-99527-0_20
4. Beyer, D., Software verification: 10th comparative evaluation (SV-COMP 2021), *Tools and Algorithms for the Construction and Analysis of Systems*, Groote, J.F. and Larsen, K.G., Eds., Lecture Notes in Computer Science, vol. 12652, Cham: Springer, 2021, pp. 401–422. https://doi.org/10.1007/978-3-030-72013-1_24
5. Beyer, D., Dangl, M., Dietsch, D., Heizmann, M., Lemberger, T., and Tautschnig, M., Verification witnesses, *ACM Trans. Software Eng. Methodology*, 1969, vol. 31, no. 4, p. 57. https://doi.org/10.1145/3477579
6. Gadelha, M.Y.R., Monteiro, F.R., Cordeiro, L.C., and Nicole, D.A., Esbmc v6.0: Verifying C programs using *k*-induction and invariant inference (competition contribution), *Tools and Algorithms for the Construction and Analysis of Systems. TACAS 2019*, Beyer, D., Huisman, M., Kordon, F., and Steffen, B., Eds., Lecture Notes in Computer Science, vol. 11429, Cham: Springer, 2019, pp. 209–213. https://doi.org/10.1007/978-3-030-17502-3_15
7. Scott, R., Dockins, R., Ravitch, T., and Tomb, A., Crux: Symbolic execution meets SMT-based verification (competition contribution), *Zenodo Preprint*, 2022. https://doi.org/10.5281/zenodo.6147218
8. Dietsch, D. and Heizmann, M., Correctness witnesses: Exchanging verification results between verifiers, *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. FSE 2016*, Seattle, WA, 2016, Beyer, D. and Dangl, M., Eds., New York: Association for Computing Machinery, 2016, pp. 326–337. https://doi.org/10.1145/2950290.2950351
9. Dietsch, D., Dangl, M., Dietsch, D., Heizmann, M., and Stahlbauer, A., Witness validation and stepwise testification across software verifiers, *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2015*, Bergamo, Italy, 2015, Beyer, D. and Dangl, M., Eds., New York: Association for Computing Machinery, 2015, pp. 721–733. https://doi.org/10.1145/2786805.2786867
10. Beyer, D. and Erkan Keremoglu, M., CPAchecker: A tool for configurable software verification, *Computer Aided Verification. CAV 2011*, Gopalakrishnan, G. and Qadeer, S., Eds., Lecture Notes in Computer Science, vol. 6806, Berlin: Springer, 2011, pp. 184–190. https://doi.org/10.1007/978-3-642-22110-1_16
11. Heizmann, M., Christ, J., Dietsch, D., Ermis, E., Hoenicke, J., Lindenmann, M., Nutz, A., Schilling, Ch., and Podelski, A., Ultimate Automizer with SMTInterpol, *Tools and Algorithms for the Construction and Analysis of Systems. TACAS 2013*, Piterman, N. and Smolka, S.A., Eds., Lecture Notes in Computer Science, vol. 7795, Berlin: Springer, 2013, pp. 641–643. https://doi.org/10.1007/978-3-642-36742-7_53
12. Khoroshilov, A., Mutilin, V., Novikov, E., Shved, P., and Strakh, A., Towards an open framework for C verification tools benchmarking, *Perspectives of Systems Informatics. PSI 2011*, Clarke, E., Virbitskaite, I., and Voronkov, A., Eds., Lecture Notes in Computer Science, vol. 7162, Berlin: Springer, 2012, pp. 179–192. https://doi.org/10.1007/978-3-642-29709-0_17
13. Novikov, E. and Zakharov, I., Verification of operating system monolithic kernels without extensions, *Leveraging Applications of Formal Methods, Verification and Validation. Industrial Practice. ISoLA 2018*, Margaria, T. and Steffen, B., Eds., Lecture Notes in Computer Science, vol. 11247, Cham: Springer, 2018, pp. 230–248. https://doi.org/10.1007/978-3-030-03427-6_19
14. Beyer, D., Verification witnesses from verification tools (SV-COMP 2023), *Zenodo Preprint*, 2023. https://doi.org/10.5281/zenodo.7627791
15. Mordan, V., Artifact: Witness structure and thoroughness analysis for SVCOMP' 2023, *Zenodo Preprint*, 2023. https://doi.org/10.5281/zenodo.15808896
16. Müller, P., West, M., and MacEachern, S., Bayesian models for non-linear autoregressions, *J. Time Ser. Anal.*, 1997, vol. 18, no. 6, pp. 593–614. https://doi.org/10.1111/1467-9892.00070
17. Aljaafari, F., Shmarov, F., Manino, E., Menezes, R., and Cordeiro, L.C., EBF 4.2: Black-box cooperative verification for concurrent programs (competition contribution), *Tools and Algorithms for the Construction and Analysis of Systems. TACAS 2023*, Sankaranarayanan, S. and Sharygina, N., Eds., Lecture Notes in Computer Science, vol. 13994, Cham: Springer, 2023, pp. 541–546. https://doi.org/10.1007/978-3-031-30820-8_33
18. Leeson, W. and Dwyer, M.B., Graves-CPA: A graph-attention verifier selector (competition contribution), *Tools and Algorithms for the Construction and Analysis of Systems*, Fisman, D. and Rosu, G., Eds., Lecture Notes in Computer Science, vol. 13244, Cham: Springer, 2022, pp. 440–445. https://doi.org/10.1007/978-3-030-99527-0_28
19. Richter, C. and Wehrheim, H., PeSCo: Predicting sequential combinations of verifiers, *Tools and Algorithms for the Construction and Analysis of Systems*, Beyer, D., Huisman, M., Kordon, F., and Steffen, B., Eds., Lecture Notes in Computer Science, vol. 11429, Cham: Springer, 2019, pp. 229–233. https://doi.org/10.1007/978-3-030-17502-3_19
20. Kettl, M. and Lemberger, T., The static analyzer infer in SV-COMP (competition contribution), *Tools and Algorithms for the Construction and Analysis of Systems*, Fisman, D. and Rosu, G., Eds., Lecture Notes in Computer Science, vol. 13244, Cham: Springer, 2022, pp. 451–456. https://doi.org/10.1007/978-3-030-99527-0_30
21. Kröning, D. and Tautschnig, M., CBMC—C bounded model checker (competition contribution), *Tools and Algorithms for the Construction and Analysis of Systems.*

TACAS 2014, Ábrahám, E. and Havelund, K., Eds., Lecture Notes in Computer Science, vol. 8413, Berlin: Springer, 2014, pp. 389–391. https://doi.org/10.1007/978-3-642-54862-8_26

22. Ayaziová, P. and Strejček, J., Symbiotic-Witch 2: More efficient algorithm and witness refutation, *Tools and Algorithms for the Construction and Analysis of Systems*, Sankaranarayanan, S. and Sharygina, N., Eds., Lecture Notes in Computer Science, vol. 13994, Cham: Springer, 2023, pp. 523–528. https://doi.org/10.1007/978-3-031-30820-8_30
23. Monat, R., Ouadjaout, A., and Miné, A., Mopsa-C: Modular domains and relational abstract interpretation for C programs (competition contribution), *Tools and Algorithms for the Construction and Analysis of Systems*, Sankaranarayanan, S. and Sharygina, N., Eds., Lecture Notes in Computer Science, vol. 13994, Cham: Springer, 2023, pp. 565–570. https://doi.org/10.1007/978-3-031-30820-8_37